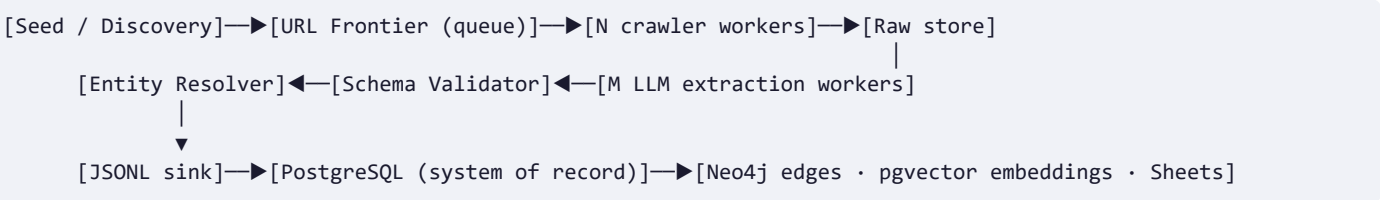
Architecture & Production Design

FrontierAtlas Ingestion Pipeline — Phase VI design document

Stack: Python 3.11+ · asyncio + aiohttp · BeautifulSoup/lxml · feedparser · Playwright (async, escalation tier) · rapidfuzz · Gemini Flash → Groq GPT-OSS 120B → DeepSeek (REST; models verified live against each provider catalog, Aug 2026) · JSONL → PostgreSQL/Neo4j/pgvector · Google Sheets export · pytest + GitHub Actions CI.

0. System Overview



Component	Implementation (trial)	Production swap	Scaling knob
URL frontier	`asyncio.Queue` per vertical	Redis Streams / SQS	queue depth, consumer count
Crawl workers	coroutines, global semaphore 32, per-domain 4	container replicas	`MAX_CONCURRENCY`, replica count
Seen-set (dedup)	append-only file, atomic `claim()`	Redis `SET NX` + TTL lease	Redis cluster shards
LLM extraction	3-tier fallback chain, per-provider token bucket	same code, more keys/tiers	RPM per tier, tier count
Validation	schema check on every record pre-write	unchanged	—
Entity resolution	seed DB + rapidfuzz, decision log	+ pgvector embedding assist	seed DB size, threshold
Sink	append-only JSONL	Postgres `COPY` batches	batch size, partitions

Every stage is **stateless against its queue** and **idempotent** (writes keyed by URL fingerprint), so the recovery strategy for any failure is uniformly *retry the unit*, and horizontal scale is purely an infrastructure change.

1. Scale Strategy — 500,000+ records without manual intervention

Discovery is decoupled from extraction. Sitemap/index/API-pagination crawlers only *emit* URLs into the frontier; detail workers only *consume*. Neither knows the other's speed — backpressure comes from bounded queues, keeping memory flat at any frontier size.

Prefer paginated APIs, fall back to sitemaps, HTML last. arXiv API (~50k records/vertical/day within politeness limits), Hugging Face Papers API, the YC company dataset, Greenhouse/Lever/Ashby boards — all cursor-resumable, so a crashed worker restarts from its checkpoint, not from zero.

Capacity math (why 500k is an infra knob, not a rewrite). One async worker at per-domain politeness 4 req/s sustained ≈ 14k pages/hour/domain. 500k records spread over ~40 source domains ≈ 12.5k/domain — **under one hour of crawling on a single 8-vCPU node** if sources allowed it; in practice politeness and anti-bot pacing dominate, so the planning number is 24–48h with 3 nodes. LLM side: ~2k input tokens/record → 1B tokens for 500k records; at three tiers × published TPM this is throughput-bounded, so the orchestrator treats tiers as *parallel capacity* (round-robin under load) rather than strictly serial fallback.

Paper→repo correlation is layered by cost: author-declared links in the abstract (deterministic regex) → the Hugging Face Papers API → GitHub citation search (repos citing the paper's arXiv ID, guarded against paper-collection repos and mega-frameworks that cite hundreds of IDs). Every link carries its provenance (`github_source`), star counts are always fetched live, and a paper without public code stays `null` — coverage is bounded by reality, never padded by guesswork.

Unattended operation = idempotency + checkpoints + the failure-mode matrix (§2.3). There is no failure class whose response is "page a human": every class maps to retry, backoff-and-retry, escalate-fetch-tier, or skip-and-log.

2. Handling 413s & 429s across thousands of concurrent extractions

2.1 — 413 / context overflow: never send an oversized payload

1. **Reduce:** HTML → dense text via lxml (scripts/nav/boilerplate stripped) — typically a 10–20× byte reduction before any budgeting.

2. **Estimate client-side:** chars/4 heuristic with a 0.9 safety margin against the *per-tier* input budget (Gemini 100k tokens, Groq 16k, DeepSeek 48k) minus prompt overhead.
3. **Truncate head+tail-biased** (70/30): titles, meta, and lead paragraphs carry the signal in news/job pages; the middle is dropped first, marked `...[truncated]...` so the model knows.
4. **Belt-and-braces:** if a provider still returns 413 (estimator too optimistic for its tokenizer), the payload is **halved and retried** — geometric convergence guarantees success in ≤3 halvings.

2.2 — 429 / rate limits: proactive pacing + reactive backoff

- **Proactive:** a token-bucket limiter per provider (aiolimiter, keyed to published RPM) smooths thousands of concurrent tasks *before* the API sees them. This is the difference between "we handle 429s" and "we rarely cause them".
- **Reactive:** exponential backoff with **full jitter** — `sleep = uniform(0, min(60s, 1s·2^attempt))` — honoring `Retry-After` (numeric or HTTP-date). Full jitter specifically prevents synchronized retry stampedes when a whole worker fleet gets limited at the same instant.
- **Fallback chain as load-shedding:** ≥3 consecutive 429s on a tier routes traffic to the next tier instead of queueing — the chain is simultaneously a *reliability* mechanism (provider outage), a *correctness* mechanism (unparseable JSON → next tier), and a *throughput* mechanism (rate pressure). The winning provider is stamped on every record (`extractedBy`) for audit.

2.3 — Failure-mode matrix (crawl + LLM)

Signal	Diagnosis	Automated response
HTTP 429 / `Retry-After`	rate limited	full-jitter backoff, honor header, ≤3 attempts → next tier/proxy
HTTP 413 / context error	payload too large	halve payload, retry (≤3 halvings)
HTTP 403/503 + challenge page	anti-bot block	rotate UA → rotate proxy → escalate to Playwright tier
HTTP 404/410	gone	skip, log, never retry
5xx / timeout / conn reset	transient	jittered retry ≤3, then requeue unit
LLM output unparseable	model drift	1 retry same tier → next tier → deterministic heuristics
All LLM tiers exhausted	keys/quota	record persists un-enriched; idempotent `enrich` backfills later

3. Freshness Tracking — never process the same item twice, across nodes

- **Canonical URL fingerprint:** lowercase scheme+host, tracking params (`utm_*`, `fbclid`, …) stripped, query keys sorted, trailing slash collapsed → SHA-256. This fingerprint is the *global* dedup key for both bulk and fresh verticals.
- **Atomic claim semantics:** the trial store is an append-only local file with `claim(fp) → bool`; the production store is Redis `SET fp NX EX <lease>` — the same one-method interface, so distribution is a backend swap. `NX` makes check-and-claim atomic: two nodes can never both win a URL. The TTL lease means a worker that crashes *after claiming but before writing* releases the URL automatically.
- **Two-phase freshness gate, before any LLM spend:** (1) RSS/API dates filter stale items pre-fetch; (2) the article page itself is re-verified — meta tags (`article:published_time`), JSON-LD `datePublished`, `<time datetime>`, then visible relative dates ("2 hours ago") normalized to UTC. Items that cannot *prove* ≤24h freshness are **dropped, never guessed** — each kept record carries `dateConfidence` (`meta | json_ld | time_tag | relative_text | rss`).
- **No-date heuristic:** date-less sources fall back to first-seen semantics on a **content hash** of extracted text (whitespace-normalized), so republished-URL tricks don't double-ingest and layout changes don't re-trigger.
- **Fuzzy-date safety:** free-text date parsing is only trusted when the text visibly contains a year or month name — otherwise parsers hallucinate dates out of strings like "Page 20 of 30" and silently poison the freshness guarantee.

4. Storage Strategy

Layer	Choice	Justification
System of record	**PostgreSQL (JSONB)**	Versioned envelope fits JSONB with real indexing: `UNIQUE(url_fingerprint)` upserts, GIN on `content->>'entityName'`, B-tree on `collectedAt`; transactional; trivially handles 500k–50M rows on one primary; `schemaVersion` enables in-place migrations.
Ingest buffer	**Append-only JSONL**	Crash-safe (a torn line is discardable, never corrupting), resumable, streams into Postgres `COPY` and the Sheets exporter; what the trial ships.

Layer	Choice	Justification
Relationship graph	**Neo4j**	The product *is* an intelligence graph. Edges: `(Startup)-[:BUILDS]->(Product)`, `(Paper)-[:IMPLEMENTED_BY]->(Repo)`, `(Startup)-[:HIRING]->(Job)`, `(News)-[:MENTIONS]->(Startup)`. Multi-hop questions ("startups whose papers trend on GitHub *and* are hiring researchers") are 3-join-hostile in SQL, native in Cypher.
Similarity assist	**pgvector (inside Postgres)**	Embedding search for near-duplicate entities and article dedup beyond exact hashing; kept in-Postgres to avoid operating a second database before scale demands it.

One source of truth: JSONL → Postgres; Neo4j and pgvector are *projections* rebuilt from Postgres at any time — no dual-write consistency problem. Entity resolution runs pre-projection so the graph only ever contains canonical nodes; the raw→canonical mapping log is itself a table (provenance + resolver regression testing).

5. Operations, Observability, Integrity

- **Metrics per vertical:** records written/dropped-invalid/dropped-stale, dedup hit-rate, fetch success by tier (plain/proxy/rendered), LLM calls by provider × outcome, 429/413 counts, backfill gap size. Exported as counters; alert on *rates* (e.g., >20% stale-drop spike = a source changed its date markup).
- **Structured logging:** namespaced (`frontieratlas.crawlers.news`), timestamped, levels env-controlled; every drop states its reason — nothing disappears silently.
- **Security:** secrets only via `.env`/environment (gitignored; `.env.example` documents shape); no keys in code or logs; robots-aware politeness caps per domain.
- **CI:** GitHub Actions runs compile + 21 unit tests (dates, dedup, resolution, chunking, backoff, schema validation) on every push.
- **Data integrity (anti-hallucination):** every record must carry a live `source.url`; LLMs only ever *restructure fetched text* — extraction prompts forbid facts not present in the payload, and records failing schema validation are dropped and counted, not repaired by guesswork.